The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

12

Discounts, Vouchers and Referrals

Part III — From Basket to Order

Taught in Laboratory	Week 9
	Geo zones and GST; a voucher applied in the wrong order
Assessment	**Assignment 1 due**

This is a class extract. It is *Chapter 12* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 12

Discounts, Vouchers and Referrals

Part 1 | Part 2 | **Part 3** | Part 4 | Part 5 | Part 6

Part III — From Basket to Order

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** the four price-reduction devices a storefront offers, and **choose** the right one for a stated objective.
- **Explain** why order totals are an ordered pipeline, and **predict** what moving a line in it does to the customer's bill.
- **Compute** the margin consequence of a discount, and **state** the extra volume needed to pay for it.
- **Describe** how a referral is tracked and paid, and **identify** what can be claimed twice.
- **Report** honestly on a feature a prescribed outline names and a platform does not provide.

Before You Start

Chapter 11: tax classes, rates and rules, and the worked order total. Chapter 10: the checkout sequence, and that an order copies its totals. A working checkout, so that discounts can be observed on real orders.

Key Terms in This Chapter

discount • quantity break • special price • coupon • coupon code • gift voucher • store credit • reward points • affiliate • referral • commission • tracking code • totals pipeline • sort order • gross margin

12.1 Four Devices, and What Each One Is For

"Give the customer money off" is one sentence and four different mechanisms, which differ in *who qualifies* and *when the shop finds out*.

Quantity break, Special, Coupon, Credit

A **quantity break** reduces the unit price above a threshold — buy ten, pay less each. It is attached to the product, applies to everyone, and the customer learns of it by filling the basket.

A **special** is a dated price that replaces the usual one for a period, for a customer group. Also attached to the product; also automatic; visible on the product page as a struck-through old price.

A **coupon** is a code the customer must type. It is attached to nothing — it is its own object, with its own dates, its own usage limits and its own restriction to particular products or categories. The code is the point: it lets the shop give a discount *to the people it gave the code to*.

Store credit is a balance on a customer's account that is spent like money. It is not a discount at all; it is a liability the shop already owes.

Which to reach for

- **To move stock** — a special. It needs no action from the customer and no code to leak.
- **To reward larger baskets** — a quantity break, which pays for itself in picking and delivery.
- **To measure a campaign** — a coupon, and *a different code per channel*. The code is the only part of a discount that tells you where the customer came from.
- **To settle a complaint** — store credit, which costs the shop its margin rather than its price list, and brings the customer back.

The mistake is to use a coupon for everything. A public code is a price cut with extra steps, and it teaches customers to search for a code before buying — which is a cost that arrives later and never goes away.

12.2 The Clause the Platform Does Not Implement

HEC's outline names "Discounts, *Vouchers*, and Referrals". A gift voucher — a code bought by one person as a stored-value instrument and spent by another — is a distinct thing from a coupon, and the shop in this course does not have one.

There are no gift vouchers in this platform

Checked directly rather than assumed:

- **No tables.** `SHOW TABLES LIKE '%voucher%'` returns nothing. The database has 159 tables and not one of them is a voucher.
- **No totals extension.** The nine shipped are `sub_total`, `shipping`, `coupon`, `tax`, `credit`, `reward`, `handling`, `low_order_fee` and `total`. No voucher.
- **One trace remains:** a demonstration image called `gift-voucher-birthday.jpg`. The feature existed in version 3 and was removed; the picture was not.

Why this is in the handout rather than quietly skipped. HEC's clause list is the contents page of a book published in 2010 (see the Course Specification), when stored-value vouchers were a standard cart feature. The outline has not been revised; the software has. Stating that plainly is more useful to you than either pretending the feature exists or pretending the clause does not.

What to do if a client asks for vouchers. Store credit (`oc_customer_transaction` and the `credit` total) already provides the spending half: a balance that draws down across orders. What is missing is the *buying* half — selling a voucher as a product, issuing a code, and letting a different person redeem it. That is a module, and Chapter 19 is where you would write it. It is the single most realistic piece of commissioned work in this course.

12.3 Totals Are a Pipeline, and Position Decides the Bill

Chapter 11 computed an order total by hand and found that the VAT line included the shipping charge. That was not arithmetic; it was *ordering*. Every total line is produced by a separate extension with a **sort order**, and each one works on the running total left by the lines before it.

Worked Example:

The same basket, with the coupon on either side of the tax. Two real orders, produced by `code/m12_discounts.py -sortorder`. The basket is Chapter 10's: one MacBook at $500.00 and two iPhones at $101.00. The coupon is the one the shop ships with, code `2222`, 10% off.

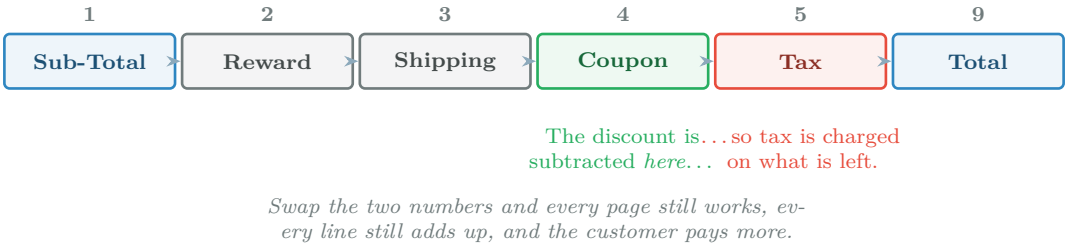


Figure 12.1: The totals pipeline as the shop ships it, with each extension's sort order above it. The numbers are editable fields in the admin panel, and nothing warns you what they mean.

As shipped — coupon at 4, tax at 5:

Sub-Total	702.00	
Flat Shipping Rate	5.00	
Coupon (2222)	−70.20	10% × 702.00
Eco Tax (-2.00)	6.00	3 × 2.00
VAT (20%)	127.36	20% × 636.80
Total	**770.16**	

The VAT base is $702.00 + 5.00 - 70.20 = 636.80$: the discount has already happened, so tax is charged on what the customer actually pays.

Coupon moved to 6, after tax:

Sub-Total	702.00	
Flat Shipping Rate	5.00	
Eco Tax (-2.00)	6.00	
VAT (20%)	141.40	20% × 707.00
Coupon (2222)	−70.20	
Total	**784.20**	

The difference is $14.04, and it is exactly 20% of the $70.20 discount — tax charged on money the customer was never asked for. On a $770 order that is 1.8%, produced by changing one integer in an admin field that offers no explanation of what it does.

Both orders are internally consistent. Both add up. Both display correctly. No validation objects, because there is nothing malformed about either — one of them is simply wrong about the world.

⚠ A sequencing rule the checkout does not tell you about

Applying a coupon *after* the customer has chosen a payment method invalidates the choice. The coupon changes the order total, the stored payment method no longer matches the recomputed list, and the final step answers:

```
{"error":"Payment method is incorrect!"}
```

— after which no order is created. The code was accepted, every earlier step reported success, and the sale is lost at the last click.

This was found by driving the sequence from a script, exactly as in Chapter 10, and it is the second fault in this course that is invisible from the browser. The remedy in your own shop is to put the coupon field *before* the method steps, or to re-select the method whenever a total changes.

12.4 What a Discount Actually Costs

A discount is not a marketing decision with a financial side effect. It is a financial decision, and the arithmetic is short enough to do before the meeting rather than after it.

Worked Example:

A product sells for **Rs 1,000** and costs the shop **Rs 700**.

Gross margin today: $1{,}000 - 700 =$ **Rs 300** per unit, 30%.

With a 10% coupon: the customer pays Rs 900; the cost is unchanged.

$$900 - 700 = \textbf{Rs 200}$$

The discount was 10% of the price and 33% of the profit.

$$\frac{300 - 200}{300} = 33.3\%$$

How much extra volume pays for it? To earn the same total profit:

$$\frac{300}{200} = 1.5$$

The shop needs 50% more orders to stand still. Not 10% more — 50%.

And that is the optimistic case, because it assumes every discounted sale is a sale that would not otherwise have happened. In practice some share of the customers who used the code would have bought anyway, and every one of those is pure loss. If half of them would have bought anyway, the required uplift roughly doubles again.

The general rule: at a gross margin of m and a discount of d (both as fractions of price), the volume multiple needed to break even is

$$\frac{m}{m - d}$$

which goes to infinity as d approaches m, and is *negative* — that is, unachievable at any volume — beyond it. A 30% discount on a 30% margin cannot be made up by selling more.

12.5 Referrals: Paying Someone Else to Send Customers

HEC's third term. The platform's mechanism is the **affiliate**: a customer account flagged as one, given a **tracking code**, and paid a commission on orders that arrive carrying it.

</> How a referral is tracked, in four steps

1. A customer applies at **Account › Affiliate** and is approved — or not, depending on **Affiliate Approval** in the settings.
2. They receive a tracking code and link their audience to `?tracking=code`.
3. A visitor arriving on that link has the code stored. The default commission in this shop is **5%**, set at **System › Settings › Edit › Option**.
4. When an order completes, a commission transaction is written against the affiliate's account, becoming store credit or a payable.

The question that decides whether this works: how long does the tracking code survive? If it lasts one request, the affiliate is paid only for customers who buy immediately.

If it lasts a month, the affiliate is paid for customers who were going to buy anyway. Neither extreme is right, and the shop — not the affiliate — must choose.

Where referral money leaks

- **Self-referral.** An affiliate buys through their own link. A 5% commission is a 5% personal discount, paid for by the shop and recorded as marketing spend.
- **Stacking with a coupon.** A 10% code plus a 5% commission is 15% off a margin that may be 30%, and half the profit is gone with two independently reasonable decisions.
- **Commission on refunded orders.** Unless the commission is reversed when the order is, the shop pays for a sale that was undone — Chapter 18's subject, and the commonest unreconciled figure in a small shop's accounts.
- **Commission on tax and shipping.** Paying 5% of $854.40 rather than 5% of the $702.00 of goods means paying the affiliate a share of money that belongs to the tax authority and the courier.

Common Pitfalls

- **Treating totals sort order as cosmetic.** It moved a bill by $14.04 and nothing warned.
- **Putting the coupon field after the payment step**, which loses the sale at the last click with an error nobody sees.
- **Assuming a demonstration coupon is usable.** The shipped codes are disabled and expired — which is correct of them, and the first thing to check when a code "does not work".
- **Approving a discount without the margin arithmetic.** 10% off a 30% margin needs 50% more orders.
- **Publishing one coupon code everywhere**, which makes the campaign unmeasurable and the discount permanent.
- **Paying commission on gross order value** rather than on goods.
- **Promising a client gift vouchers** because the outline mentions them. This platform has none; it is a module, not a setting.

Try It Yourself: Move one number, and change what the customer pays

```
# --- 1. Find out why the shipped coupon does not work ---------------
cd code
docker compose exec db mariadb -ushop -pshoppw opencart -e \
  "SELECT code,name,status,date_start,date_end FROM oc_coupon;"
# Two reasons, not one. Name both before fixing either.

# --- 2. Enable it ---------------------------------------------------
python m12_discounts.py --coupon on

# --- 3. Place one order with it and take the total apart ------------
python m12_discounts.py --order
# Reproduce every line with a calculator.

# --- 4. Predict, in writing -----------------------------------------
# If the coupon were applied AFTER tax instead of before, would the
# customer pay more, less, or the same? By how much? Commit first.

# --- 5. Run the experiment ------------------------------------------
python m12_discounts.py --sortorder

# --- 6. Account for the difference exactly --------------------------
```

```
# It is not an arbitrary number. Show where it comes from.

# --- 7. Build a measurable campaign ---------------------------------
# Admin: Marketing > Coupons. Make THREE codes for the same 10% offer,
# one per channel. Say how you would later tell which worked.

# --- 8. Do the margin arithmetic before you would ship it -----------
# Your product costs 700 and sells for 1000. Work out the volume
# uplift your 10% code needs. Then do it for 20%.

# --- 9. Put everything back -----------------------------------------
python m12_discounts.py --restore
```

What to notice. Step 4 against step 5. Most people predict "the same" — the discount and the tax are both there either way. The 1.8% difference is the whole chapter, and it lives in a numeric field with no label explaining it.

Checkpoint — Test Yourself

1. The same basket produced totals of 770.16 and 784.20 depending on one setting. Explain what the setting was, why the difference is exactly 14.04, and which total is correct — giving your reason in terms of what tax is owed on.
2. A client with a 25% gross margin wants to run a permanent 15% "loyal customer" code. Work out the volume uplift needed to break even, state two reasons the real requirement is higher than your figure, and propose an alternative that costs less.
3. HEC's outline requires vouchers and the platform has none. Describe what you would tell a client who asked for them, what the platform already provides, and what specifically you would have to build.

Chapter Summary

- Four devices, differing in who qualifies and when the shop learns of it: quantity breaks and specials are automatic and attached to products; a coupon is a code, which is what makes a campaign measurable; store credit is a liability, not a discount.
- This platform has **no gift vouchers** — no tables, no totals extension, only a leftover demonstration image. HEC's clause reflects a 2010 book, not current software. Say so rather than fudging it; the gap is a module, and Chapter 19 is where it would be written.
- Order totals are an ordered pipeline. Each line works on what the previous lines left, and every extension's position is an editable integer with no explanation attached.
- Worked: the same basket totalled 770.16 with the coupon before tax and 784.20 with it after — a difference of 14.04, exactly 20% of the 70.20 discount. Both are internally consistent; one is wrong about what tax is owed on.
- Applying a coupon after the payment method is chosen invalidates the choice and loses the order at the last click, with an error the browser does not show.
- Worked: a 10% discount on a 30% margin removes 33% of the profit and needs 50% more orders to break even — $m/(m-d)$ in general, and unachievable once d reaches m.
- A referral is tracked by a code and paid as commission. The leaks are self-referral, stacking with coupons, commission on refunded orders, and commission on tax and shipping.

Review Questions

1. (MCQ) Moving the coupon line after the tax line in the totals pipeline causes the customer to pay: (a) less; (b) the same; (c) more, by the tax on the discount; (d) more, by the discount.

2. **(MCQ)** A 10% discount on a product with a 30% gross margin removes what share of the profit? (a) 10%; (b) about a third; (c) half; (d) it depends on the volume.

3. **(Short)** Give two things a coupon can do that a special cannot, and one thing a special does better.

4. **(Short)** Explain why a shop should issue a different coupon code for each channel, even for the same offer.

5. **(Short)** Explain why paying affiliate commission on the order grand total rather than on the goods value is a mistake, using the worked order in Chapter 11 for your figures.

6. **(Applied)** A shop runs a 20% code and a 5% affiliate scheme on products with a 35% gross margin. Work out the margin on an order that uses both, say what the shop should do about it, and state how you would enforce your answer in the platform.

7. **(Applied)** Design the gift voucher the platform lacks. State the tables you would add, how a voucher is bought, how it is redeemed by a different person, what happens when it part-pays an order, where it sits in the totals pipeline and why, and the two ways it could be spent twice.